

Quantifying the Effect of Quantum Operations in Machine Learning by Comparison to Classically Simulable Variants

Víctor Mauricio Gutiérrez Luque

12.05.2026

Computations were performed with computing resources granted by RWTH Aachen University under project **thes2204**.

Abstract

Quantum machine learning (QML) investigates whether quantum-mechanical representations and computations can improve machine-learning tasks. However, it is often unclear which observed performance differences are genuinely attributable to specifically quantum features rather than to architectural differences that can be implemented classically or benchmarking effects. This thesis studies the role of entanglement in variational QML models through a controlled ablation or removal framework. The work introduces partially and fully separable variants of the Circuit-Centric Classifier (CCC) and IQP-Kernel Classifier, constructed by systematically reducing entangling interactions while keeping the remaining training and evaluation pipeline fixed. The models are evaluated on multiple synthetic and structured benchmark families, including but not limited to linearly separable, hidden-manifold, and hyperplane-based datasets. The results show that partially entangling variants often remain competitive with the original models, whereas fully separable variants typically exhibit larger performance degradation. The experiments are consistent with entanglement contributing meaningfully to performance in the studied models, while also suggesting that sparse entangling connectivity may already retain much of the observed benefit of the original architectures in this benchmark setting. The IQP-kernel experiments scaled prohibitively in higher-dimensional settings, exceeding the available computational budget.

Contents

Abstract	1
1 Introduction and Related Literature	4
2 Background	6
2.1 Quantum Computing Preliminaries	6
2.1.1 Historical Background	6
2.1.2 Some Linear Algebra	8
2.1.3 Postulates of Quantum Mechanics	8
2.1.4 Quantum Computing	10
2.2 Classical Machine Learning	15
2.2.1 Linear Classifiers	15
2.2.2 Support Vector Machine (SVM)	16
2.2.3 Kernel Methods	16
2.2.4 Neural Networks (MLPs)	17
2.3 Quantum Machine Learning	17
2.3.1 How is data encoded?	17
2.3.2 Ansatz: How is the model learning?	19
2.3.3 Models in Quantum Machine Learning	19
2.4 Motivation: The Benchmarking Problem in QML	21
3 Methodology	24
3.1 Experimental Setup	24
3.2 Datasets	25
3.3 Circuit-Centric Classifier	27
3.3.1 Input Encoding	27
3.3.2 Variational Circuit and Prediction Rule	28
3.4 Entanglement Ablation	28
3.4.1 Controlled Ansatz	29
3.4.2 CCC Variants	31
3.5 IQP Kernel Classifier	32

3.6	Entanglement Ablation	33
3.6.1	IQP Kernel Variants	34
3.7	Training Procedure	35
3.8	Evaluation	36
3.9	Reproducibility	36
4	Results	38
4.1	Quantitative Results	38
4.2	Qualitative Interpretation	43
4.3	Limitations	45
5	Conclusion and Outlook	47

Chapter 1

Introduction and Related Literature

Quantum machine learning sits at the intersection of quantum computing and statistical learning, and much of the literature has been motivated by the possibility that quantum models may represent or process patterns in ways that are classically difficult to reproduce. Foundational work on quantum feature maps, variational quantum circuits, and quantum kernels established the main model classes now used in the field (Mitarai et al., 2018; Schuld et al., 2019; Havlíček et al., 2019; Benedetti et al., 2019). At the same time, more recent critical work has emphasized that the central scientific question is not simply whether a quantum model can outperform a classical baseline on a particular benchmark, but whether the observed behavior can actually be attributed to specifically quantum ingredients rather than to tuning choices, data selection, or architectural details that could also be reproduced classically (Bowles et al., 2024).

This thesis is positioned within that second line of work. Rather than treating the presence of a quantum circuit as sufficient evidence of quantum advantage, it adopts an ablation-based view in which quantum models are compared to dequantized or classically simulable variants that differ in one controlled architectural feature. The focus here is entanglement. Entanglement is often regarded as one of the defining nonclassical resources of quantum computation, but in practical QML models it is not always clear whether it is genuinely responsible for predictive performance, or whether similar results can already be obtained with sparse or even fully local structures. The methodology of this thesis is therefore to reduce entanglement systematically while keeping the rest of the pipeline fixed.

The remainder of the document is structured as follows. Chapter 2 develops the background needed for the later analysis, including the ba-

sis quantum-computing formalism, core classical machine-learning concepts, the main QML model classes, and the benchmarking problem that motivates the study. Chapter 3 presents the experimental methodology, describes the benchmark families, defines the original and dequantized CCC variants, and outlines the corresponding IQP-kernel ablations together with the shared training and evaluation pipeline. Chapter 4 reports the empirical results, first quantitatively through ranking, forest, and accuracy plots, and then qualitatively through benchmark-family-level interpretation. Chapter 5 concludes by summarizing what the experiments show about entanglement in the studied models and by discussing possible directions for further work.

Chapter 2

Background

2.1 Quantum Computing Preliminaries

2.1.1 Historical Background

In the last century, there were measured results in classical physics giving absurdities. Initially, when people studied how hot objects glowed, the old theory predicted they should emit infinite amounts of UV light. Planck resolved this problem by proposing that energy is not exchanged continuously, but in discrete packets called quanta (Planck, 1900). Subsequently, the photoelectric effect showed that light can knock electrons out of metal only if its frequency is high enough; brightness or intensity of light alone did not help. Einstein explained this phenomenon by saying light behaves like particles (photons), each carrying one energy packet (Einstein, 1905). A further development was Bohr's contribution. Atoms when heated were seen to emit only specific wavelengths or colors (spectral lines) and not a smooth rainbow. This made sense if electrons inside atoms can only have certain allowed energies. Bohr applied the quantum idea to atoms, proposing quantized energy levels for electrons in 1913, but without explaining why (Bohr, 1913). Consequently, De Broglie suggested that not only light, but also matter like electrons can act like waves; electron-diffraction experiments confirmed this (de Broglie, 1924).

Heisenberg and Schrödinger completed it by independently constructing new, self-consistent replacements for classical mechanics. Heisenberg started from the experimental fact that atomic spectra reveal only transitions between discrete energy levels. This led naturally to a matrix-based algebra in which the order of operations matters, capturing quantum behavior without relying on classical orbits. Schrödinger, in contrast, searched for a wave equation governing microscopic systems of matter. This yielded cor-

rect bound-state energies and spectral predictions. They were soon shown to be mathematically equivalent descriptions of the same theory, and Born provided the probabilistic interpretation of the wavefunction (Heisenberg, 1925; Schrödinger, 1926; Born, 1926). In short: classical physics failed for small-scale phenomena, and quantum mechanics was the new rulebook that fit the experiments.

In the classical computation model, information is encoded in bits and processed by deterministic or randomized operations, and the time and memory required by these operations determine what is computationally feasible. Quantum computing extends this model by introducing a new unit of information, the qubit, whose behavior follows the rules of quantum mechanics. Unlike a classical bit, which is always either 0 or 1, a qubit can occupy infinitely many states on a continuous state space; before measurement it can exist in a superposition of basis states; measurement irreversibly changes the qubit’s state; and the outcome depends on the measurement basis, meaning the same qubit can be probed along many different axes (Nielsen and Chuang, 2010).

These features are valuable not because they allow unlimited extraction of information, but because they allow information to be encoded in complex amplitudes and phases, manipulated through continuous unitary transformations, and combined across qubits to produce interference and entanglement.

Well-designed quantum algorithms exploit these effects so that, after a sequence of gates, the probability of measuring a desired result is amplified while unwanted outcomes are suppressed. These capabilities have no direct classical analogue. This mechanism is behind landmark quantum speedups such as Shor’s algorithm for integer factoring and discrete logarithms, which runs in polynomial time on a quantum computer compared to the best known sub-exponential classical methods (Shor, 1994, 1997), and Grover’s algorithm for unstructured search, which reduces $O(N)$ classical query complexity to $O(\sqrt{N})$ using amplitude amplification (Grover, 1996).

We will now first introduce the minimal linear-algebra language needed to describe quantum states and operations; then define qubits and single-qubit gates; extend to multi-qubit systems via tensor products and entanglement; and finally present the circuit model as the formal framework for quantum computation.

2.1.2 Some Linear Algebra

We briefly review the basic linear algebra concepts used throughout this thesis.¹

A vector is a geometric object that has a magnitude and a direction and can live in a certain vector space, represented as columns whose values are in the domain. The basic operations are vector addition and scalar multiplication, and any expression of the form $\sum_i \alpha_i v_i$ is a linear combination of vectors v_i with coefficients α_i .

A map between vector spaces is linear if it respects these operations, $L(\alpha v + \beta w) = \alpha L(v) + \beta L(w)$. The standard Hermitian inner product is $\langle v, w \rangle = \sum_i \bar{v}_i w_i$, which induces the norm $\|v\| = \sqrt{\langle v, v \rangle}$. Two vectors are orthogonal if $\langle v, w \rangle = 0$, and a vector is normalized if $\|v\| = 1$. We often write vectors and inner products in bra-ket notation, where a column vector is denoted $|v\rangle$, its conjugate transpose is $\langle v|$, and the inner product is $\langle v|w\rangle$.

A basis $\{e_1, \dots, e_n\}$ of $\mathbb{C}^n$ is a set of vectors such that every vector has a unique coordinate representation $v = \sum_i \alpha_i e_i$; if, in addition, $\langle e_i, e_j \rangle = \delta_{ij}$, the basis is orthonormal. The standard (computational) basis of $\mathbb{C}^n$ consists of the unit vectors with a single 1 in one position and 0 elsewhere, and any vector can be written as a linear combination of these basis vectors.

Linear maps on the vector space are represented by matrices, and their action on a vector is given by matrix-vector multiplication; special directions that are only rescaled by a matrix are captured by eigenvectors v with eigenvalues λ , satisfying $Av = \lambda v$. Of particular importance are unitary matrices U , defined by the condition $U^\dagger U = I$, where $U^\dagger$ is the conjugate transpose and I is the identity. Unitary operators preserve inner products, and hence norms and angles, and are always reversible with inverse $U^{-1} = U^\dagger$.

2.1.3 Postulates of Quantum Mechanics

In this subsection we summarize the standard postulates of finite-dimensional quantum mechanics in the language of complex vector spaces introduced above. According more detailed discussion can be found in Nielsen and Chuang (2010, Chapter 2).

Postulate 1 (State space). To every isolated physical system there corresponds a complex Hilbert space $\mathcal{H}$, called the state space of the system. The (pure) states of the system are represented by unit vectors $|\psi\rangle \in \mathcal{H}$ with $\langle \psi | \psi \rangle = 1$. Two vectors that differ only by a nonzero complex scalar multiple

¹This overview is based on the exposition in Nielsen and Chuang (2010, Chapter 2).

represent the same physical state. In particular, $|\psi\rangle$ and $e^{i\theta}|\psi\rangle$ are physically indistinguishable for any real θ .

Postulate 2 (Time evolution of closed systems). The time evolution of a closed system is described by a unitary operator on its state space. If the state at time t_1 is $|\psi(t_1)\rangle$ and the state at time t_2 is $|\psi(t_2)\rangle$, then there exists a unitary operator U on $\mathcal{H}$ such that

$$|\psi(t_2)\rangle = U|\psi(t_1)\rangle, \quad (2.1)$$

where $U^\dagger U = I$. Unitary evolution preserves inner products and, in particular, the normalization of state vectors.

Postulate 3 (Measurement). Let $\{|i\rangle\}$ be an orthonormal basis of the state space $\mathcal{H}$, and consider a measurement in this basis. Any state can be written as

$$|\psi\rangle = \sum_i \alpha_i |i\rangle, \quad \sum_i |\alpha_i|^2 = 1. \quad (2.2)$$

The measurement has discrete outcomes labeled by i , and quantum mechanics assigns:

- probability

$$p(i) = |\alpha_i|^2 = |\langle i|\psi\rangle|^2 \quad (2.3)$$

to obtaining outcome i ;

- post-measurement state

$$|\psi_i\rangle = |i\rangle \quad (2.4)$$

conditioned on observing outcome i .

Equivalently, this measurement can be described by the family of orthogonal projectors $P_i = |i\rangle\langle i|$, with $p(i) = \langle\psi|P_i|\psi\rangle$ and normalized post-measurement states $|\psi_i\rangle = P_i|\psi\rangle/\sqrt{p(i)}$. More general measurements can be modeled by a collection of measurement operators $\{M_m\}$ satisfying $\sum_m M_m^\dagger M_m = I$, but in this thesis we will primarily use projective measurements in the computational basis.

Postulate 4 (Composite systems). For a composite system consisting of subsystems A and B with state spaces $\mathcal{H}_A$ and $\mathcal{H}_B$, the state space of the combined system is the tensor product

$$\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B. \quad (2.5)$$

If $|\psi\rangle_A \in \mathcal{H}_A$ and $|\phi\rangle_B \in \mathcal{H}_B$ are states of the individual subsystems, then the product state of the composite system is $|\psi\rangle_A \otimes |\phi\rangle_B$. More generally, a pure state of the composite system is any unit vector $|\Psi\rangle_{AB} \in \mathcal{H}_A \otimes \mathcal{H}_B$, which can be expressed as

$$|\Psi\rangle_{AB} = \sum_{ij} \alpha_{ij} |i\rangle_A \otimes |j\rangle_B \quad (2.6)$$

with respect to chosen orthonormal bases $\{|i\rangle_A\}$ and $\{|j\rangle_B\}$. A pure state is called *separable* if it can be written as $|\Psi\rangle_{AB} = |\psi\rangle_A \otimes |\phi\rangle_B$ for some $|\psi\rangle_A$ and $|\phi\rangle_B$, otherwise *entangled*. Even when entangled qubits are spatially separated, a measurement on one qubit instantaneously fixes the conditional state of the others: once an outcome is obtained on one side, the outcome probabilities on the other side are updated accordingly. For example, if

$$|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix}, \quad |\phi\rangle = \begin{pmatrix} c \\ d \end{pmatrix},$$

then their tensor product is:

$$|\psi\rangle \otimes |\phi\rangle = \begin{pmatrix} ac \\ ad \\ bc \\ bd \end{pmatrix}.$$

2.1.4 Quantum Computing

Classical computers represent information using *bits*, which can take values of either 0 or 1. Quantum computing generalizes this idea using *quantum bits*, or *qubits*.

A single qubit is a two-dimensional quantum system with state space $\mathcal{H} \cong \mathbb{C}^2$. Fixing the computational basis $\{|0\rangle, |1\rangle\}$, any pure state of a qubit can be written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (2.7)$$

As global phases are irrelevant, because $|\psi\rangle$ and $e^{i\theta}|\psi\rangle$ describe the same physical state, we can use this to parametrize any qubit state by two real angles $\theta \in [0, \pi]$ and $\varphi \in [0, 2\pi)$ as

$$|\psi(\theta, \varphi)\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\varphi} \sin\left(\frac{\theta}{2}\right) |1\rangle. \quad (2.8)$$

The pair (θ, φ) can be interpreted as spherical coordinates of a point on the unit sphere in $\mathbb{R}^3$. This represents the *Bloch sphere*: every qubit state

corresponds to a point on the surface of the sphere, with $|0\rangle$ and $|1\rangle$ at the north and south poles, respectively.

Unitary operators preserve inner products and norms, so they map pure states to pure states, corresponding to rotations.

A *single-qubit gate* is such a unitary operation applied to an individual qubit. Important examples include the Pauli operators

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad (2.9)$$

which generate π -rotations about the x -, y -, and z -axes of the Bloch sphere. The Pauli- X gate exchanges $|0\rangle$ and $|1\rangle$ and is the quantum analogue of a classical NOT gate. The Pauli- Z gate leaves $|0\rangle$ invariant and multiplies $|1\rangle$ by a phase -1 , thereby changing relative phase without altering the measurement probabilities in the computational basis.

Another fundamental gate is the Hadamard operator

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad (2.10)$$

which maps basis states to equal superpositions,

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}. \quad (2.11)$$

The Hadamard gate is routinely used to create superposition states from computational basis states. More generally, one often considers continuous families of *rotation gates*

$$R_X(\theta) = e^{-i\theta X/2}, \quad R_Y(\theta) = e^{-i\theta Y/2}, \quad R_Z(\theta) = e^{-i\theta Z/2}, \quad (2.12)$$

which implement rotations of the Bloch vector by an angle θ around the corresponding axis, as well as phase gates such as

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \quad (2.13)$$

which modify the relative phase between $|0\rangle$ and $|1\rangle$.

To describe multiple qubits within the formalism of Postulate 4, we use the tensor product to build composite systems. If each single qubit lives in a two-dimensional Hilbert space $\mathcal{H} \cong \mathbb{C}^2$ with basis $\{|0\rangle, |1\rangle\}$, then the state space of an n -qubit register is the tensor product

$$\mathcal{H}^{\otimes n} = \underbrace{\mathcal{H} \otimes \cdots \otimes \mathcal{H}}_{n \text{ times}} \cong \mathbb{C}^{2^n}. \quad (2.14)$$

The computational basis of $\mathcal{H}^{\otimes n}$ consists of all 2^n tensor products of single-qubit basis states,

$$\{|x_1\rangle \otimes \cdots \otimes |x_n\rangle : x_k \in \{0, 1\}\}, \quad (2.15)$$

which are usually written more compactly as $|x_1 \cdots x_n\rangle$ for $x_1 \cdots x_n \in \{0, 1\}^n$. Any pure n -qubit state can be expressed as a linear combination

$$|\Psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1. \quad (2.16)$$

A particularly simple class of multi-qubit states are *product states*, which factorize as a tensor product of single-qubit states, for example

$$|\Psi\rangle = |\psi_1\rangle \otimes \cdots \otimes |\psi_n\rangle, \quad (2.17)$$

with each $|\psi_k\rangle = \alpha_k|0\rangle + \beta_k|1\rangle$. However, Postulate 4 allows more general states that cannot be written in this factorized form, entangled states. Popular examples are the Bell states, such as

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle). \quad (2.18)$$

Measuring the first qubit of $|\Phi^+\rangle$ in the computational basis yields outcome 0 or 1 with equal probability, but conditioned on the outcome, the state of the second qubit is perfectly correlated: if the first outcome is 0 the joint state collapses to $|00\rangle$, whereas if the first outcome is 1 it collapses to $|11\rangle$. These correlations cannot be explained by any classical product state and are stronger than those obtainable from independent systems with shared classical randomness.

Entanglement plays a central role in quantum computation because it allows a quantum computer to represent and manipulate correlations across many subsystems in a way that has no direct classical analogue. In an n -qubit register, generic pure states live in a space of dimension 2^n , and entangled states can encode joint amplitude patterns over all 2^n basis strings.

It is important to distinguish entanglement from superposition. Superposition already appears at the single-qubit level: a state of the form $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ is a superposition of basis states, but not entangled, because it concerns only one system. Every entangled state is also a superposition, but not every superposition is entangled.

A quantum circuit consists of a sequence of quantum gates acting on a register of qubits. The structure of a quantum circuit reflects the algorithmic logic of the computation, with qubits initialized in known basis states,

processed through a network of gates, and finally measured to extract classical information. Quantum algorithms exploit this structure by applying multi-qubit gates that create and transform entanglement, so that interference between many computational paths can be steered towards desired outcomes.

Grover's Algorithm

To illustrate the operation of a quantum circuit, consider Grover's algorithm (Grover, 1996), which provides a quadratic speedup for the problem of searching an unstructured database. Grover's algorithm works as followed: given a function $f : \{0,1\}^n \rightarrow \{0,1\}$ that evaluates to $f(x) = 1$ for a unique input $x = x_0$, and $f(x) = 0$ otherwise, the goal is to find x_0 . Classically, this requires $\mathcal{O}(2^n)$ evaluations in the worst case, whereas Grover's algorithm achieves this in $\mathcal{O}(\sqrt{2^n})$ steps.

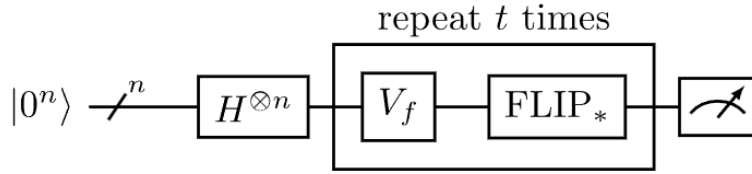


Figure 2.1: The quantum circuit for Grover's algorithm

1. Initialization

We begin with an n -qubit register initialized to the state $|0\rangle^{\otimes n}$. Applying the Hadamard transform $H^{\otimes n}$ yields the uniform superposition:

$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \{0,1\}^n} |x\rangle, \quad \text{where } N = 2^n.$$

This state assigns equal probability amplitude to every possible input $x \in \{0,1\}^n$.

2. Oracle Operator

The oracle V_f is a unitary operator that flips the sign of the amplitude of the solution x_0 . Formally:

$$V_f|x\rangle = \begin{cases} -|x\rangle & \text{if } x = x_0, \\ |x\rangle & \text{otherwise.} \end{cases}$$

This phase inversion does not reveal x_0 , but encodes its identity into the quantum state.

3. Diffusion Operator or FLIP_{*}

The diffusion operator, often referred to as the inversion about the average, amplifies the probability amplitude of the solution. It is defined as:

$$D = 2|\psi\rangle\langle\psi| - I,$$

where $|\psi\rangle$ is the initial uniform superposition and I is the identity matrix. When applied to a state $|\phi\rangle$, the operator reflects $|\phi\rangle$ about $|\psi\rangle$.

4. Grover Iteration

One Grover iteration consists of applying the oracle followed by the diffusion operator:

$$G = D \cdot V_f.$$

The state after t iterations is:

$$|\psi^{(t)}\rangle = G^t|\psi\rangle.$$

It can be shown that each application of G rotates the state vector closer toward the solution state $|x_0\rangle$ in a two-dimensional subspace spanned by $|x_0\rangle$ and its orthogonal complement. The rotation angle θ satisfies:

$$\cos(\theta) = \sqrt{\frac{N-1}{N}}, \quad \text{so } \theta \approx \frac{1}{\sqrt{N}} \text{ for large } N.$$

To maximize the probability of measuring x_0 , the number of iterations should be approximately:

$$t = \left\lfloor \frac{\pi}{4} \sqrt{N} \right\rfloor.$$

5. Measurement and Success Probability

After applying t iterations, we measure the quantum state in the computational basis. With high probability (approaching 1 as $N \rightarrow \infty$),

the outcome will be the desired value x_0 . The overall complexity of the algorithm is therefore $\mathcal{O}(\sqrt{N})$, providing a quadratic speedup over classical brute-force search.

2.2 Classical Machine Learning

Supervised learning is a core approach in machine learning in which the goal is to infer a mapping from inputs to outputs using a dataset of labeled examples. Each example is represented as a pair $(\mathbf{x}, y)$, where $\mathbf{x} \in \mathbb{R}^n$ is an input vector composed of n features, and y is the corresponding target value. The learning algorithm or mathematical function, the model, attempts to approximate a function $f : \mathbb{R}^n \rightarrow \mathcal{Y}$ that best predicts y given $\mathbf{x}$, where $\mathcal{Y}$ denotes the set of all possible target values, also called the output or label space (Murphy, 2012).

Some models in supervised learning are called *parametric*, meaning they assume a fixed form for the function f and learn a set of parameters, typically denoted by a weight vector $\mathbf{w} \in \mathbb{R}^n$ and a bias term $b \in \mathbb{R}$ during training. Others are *non-parametric*, which means their structure or complexity grows with the amount of data, without assuming a fixed number of parameters (Murphy, 2012).

In classification tasks, models often produce an intermediate output $z = \mathbf{w}^\top \mathbf{x} + b$, called the score. This is a real-valued scalar that serves as a bridge between raw input features through a decision or *activation function* to final output labels $\hat{y} \in \mathcal{Y}$. For example, a *sign function* $\hat{y} = \text{sign}(z)$ returns $+1$ or -1 based on the sign of z , while probabilistic models may use sigmoid or softmax functions to produce class probabilities. The geometry and dimensionality of the feature space (i.e., how many and which features are used to describe each $\mathbf{x}$) deeply influence how models learn and generalize.

2.2.1 Linear Classifiers

Linear classifiers are a class of models that separate data points in a feature space using a hyperplane defined by a weight vector $\mathbf{w} \in \mathbb{R}^n$ and a scalar bias term $b \in \mathbb{R}$. The model computes a linear combination $z = \mathbf{w}^\top \mathbf{x} + b$, and the sign of z determines the predicted class. This leads to a decision function of the form $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$, where $\text{sign}(\cdot)$ returns $+1$ for positive inputs and -1 for negative ones (Rosenblatt, 1958).

The vector $\mathbf{w}$ encodes the relative importance of each feature in the input $\mathbf{x}$, while the bias b shifts the decision boundary. The decision boundary itself is the set of points $\mathbf{x}$ for which $\mathbf{w}^\top \mathbf{x} + b = 0$. Linear classifiers rely on

the assumption that classes can be separated by such a boundary in the input space (Rosenblatt, 1958). They form the basis of many more complex models and are computationally simple, making them a foundational concept in machine learning.

2.2.2 Support Vector Machine (SVM)

While linear classifiers separate the classes by finding any line, which makes them sensitive to outliers, support vector machines (SVMs) (Cortes and Vapnik, 1995), by construction, are margin-based classifiers that aim to find the hyperplane separating two classes, and not focus on outlier points. This makes them more outlier-robust. Given training examples labeled $y_i \in \{-1, +1\}$, the SVM solves an optimization problem that finds the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ such that the distance between this hyperplane and the closest data points (called *support vectors*) is maximized.

Maximizing the margin improves the model’s robustness to variations in the data. The supporting hyperplanes, those defining the margin boundaries, are given by $w^\top x + b = \pm 1$. The distance from the decision boundary to each of these hyperplanes is $\frac{1}{\|w\|}$. Therefore, the total margin width, which is the distance between the two margin hyperplanes, is $\frac{2}{\|w\|}$. Maximizing it is equivalent to minimizing $\|\mathbf{w}\|^2$, which leads to the primal optimization formulation. This choice regularizes the model by penalizing large weights, which in turn reduces sensitivity to small fluctuations in input values.

The model predicts labels using the *sign function*: $f(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$. In this setting, the sign of the inner product determines on which side of the hyperplane the input lies. If the data are not linearly separable, SVMs can be extended using kernel functions, which allow the model to behave as if the data were mapped into a higher-dimensional space, where a linear separator may exist, without explicitly computing that mapping.

2.2.3 Kernel Methods

Kernel methods (Schölkopf and Smola, 2002) are a set of techniques that allow linear models to learn non-linear patterns by computing similarities between data points using a *kernel function*. A kernel is a function $K(\mathbf{x}, \mathbf{x}')$ that measures how similar two inputs are, without explicitly mapping them into a higher-dimensional space. Instead, it leverages the mathematical property that inner products in this high-dimensional space can be computed directly via the kernel, a concept known as the *kernel trick*.

One widely used kernel is the Gaussian kernel or RBF (Radial Basis Function):

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

This function assigns high similarity to points that are close in Euclidean space and low similarity to distant points. This similarity measure enables the model to construct complex, non-linear decision boundaries in the input space, while still solving a linear problem in an implicit feature space.

2.2.4 Neural Networks (MLPs)

Multilayer Perceptrons (MLPs) (Goodfellow et al., 2016) are a class of feed-forward neural networks consisting of multiple layers of *artificial neurons*. Each neuron computes a weighted sum of its inputs, applies a non-linear *activation function* such as the rectified linear unit (ReLU), sigmoid, or hyperbolic tangent (tanh), and passes the result to the next layer. An MLP typically consists of an *input layer*, one or more *hidden layers*, and an *output layer*.

Formally, each hidden layer performs a transformation of the form:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases of layer l , and $\sigma(\cdot)$ is the activation function. The model is trained using *backpropagation*, which computes gradients of a loss function with respect to each parameter using the chain rule, followed by *gradient descent* to update the weights.

Small MLPs, typically with one or two hidden layers and a moderate number of units, are capable of learning complex, non-linear relationships while maintaining tractability. Unlike kernel methods or instance-based models, MLPs *learn internal representations* of the data during training, which makes them a flexible and general-purpose modeling approach.

2.3 Quantum Machine Learning

2.3.1 How is data encoded?

Classical data can be embedded into quantum states in several ways; here we briefly summarise a few common schemes following Rath and Date (2023).

Basis (computational) encoding. The most direct method maps each classical bit to a qubit in the computational basis. A classical bit string such as 101 is represented as the basis state

$$|101\rangle = |1\rangle \otimes |0\rangle \otimes |1\rangle.$$

If we measure in the computational basis, we always obtain the string 101; the information is stored in *which* basis state the system occupies.

Superposition encoding. Instead of a single basis state, one can encode information in a superposition of several classical strings. For example,

$$|\psi\rangle = \frac{1}{\sqrt{3}}(|100\rangle + |010\rangle + |001\rangle)$$

encodes that the system is in one of the three strings 100, 010, or 001 with equal probability. Measurement yields one of these outcomes with probability $1/3$, so the information is contained in which basis states appear and with which amplitudes.

Angle encoding. In angle (or rotation) encoding, real-valued features are used as rotation angles on single qubits. For a two-dimensional data point $x = (x_1, x_2)$, one can prepare

$$|\psi(x)\rangle = R_y(x_1) \otimes R_y(x_2) |00\rangle = R_y(x_1)|0\rangle \otimes R_y(x_2)|0\rangle,$$

where $R_y(x)$ is a rotation about the y -axis by angle x . The measurement probabilities then depend smoothly on the input values x_1 and x_2 . Analogy: each qubit is an arrow on the Bloch sphere. Your data tells you how much to turn the arrow.

Amplitude encoding. Amplitude encoding uses the components of a (normalised) classical vector as the amplitudes of a quantum state. For instance, for $x = (1, 2, 0, 1)$ we first normalise $\|x\| = \sqrt{1^2 + 2^2 + 0^2 + 1^2} = \sqrt{6}$ and prepare the two-qubit state

$$|\psi(x)\rangle = \frac{1}{\sqrt{6}}|00\rangle + \frac{2}{\sqrt{6}}|01\rangle + 0 \cdot |10\rangle + \frac{1}{\sqrt{6}}|11\rangle.$$

Here the classical information is encoded directly in the probability amplitudes of each basis state. The classical numbers (1,2,0,1) decide how likely each basis state is, after the normalisation.

2.3.2 Ansatz: How is the model learning?

After encoding, the quantum state is processed by a variational ansatz, i.e. a quantum circuit $U_{\text{var}}(\theta)$ with a fixed gate layout and fixed parameters or trainable angles θ . The ansatz specifies how many layers of single-qubit rotations and entangling gates are used, on which qubits they act, and how the qubits are connected. In this sense, it plays a role analogous to the architecture of a classical neural network: choosing the number of layers, the width of each layer, and the connectivity between neurons. While the feature map determines how the classical input x is embedded into the Hilbert space, the ansatz provides the trainable degrees of freedom that are optimized during learning and thus defines the family of functions that the model can represent (Benedetti et al., 2019).

2.3.3 Models in Quantum Machine Learning

Following the models introduced in (Bowles et al., 2024), we now discuss the following:

VQCs: Variational Quantum Circuits

In the setting of quantum machine learning, variational quantum circuits or VQC's are often described abstractly as functions of the form

$$f(\theta, x) = \text{tr}[O(x, \theta) \rho(x, \theta)],$$

where $\rho(x, \theta)$ is a density matrix and $O(x, \theta)$ is an observable. This expression can be understood operationally as a three-step procedure. First, a feature map (encoding circuit) $U_{\text{enc}}(x)$ prepares a data-dependent quantum state from a simple initial state ρ_0 , for example $|0 \dots 0\rangle\langle 0 \dots 0|$. This yields the encoded state $U_{\text{enc}}(x) \rho_0 U_{\text{enc}}(x)^\dagger$. Second, a variational ansatz $U_{\text{var}}(\theta)$, i.e. a parametrized quantum circuit with fixed gate layout and trainable angles θ , is applied to this encoded state. The result is the joint data- and parameter-dependent state, or the density matrix,

$$\rho(x, \theta) = U_{\text{var}}(\theta) U_{\text{enc}}(x) \rho_0 U_{\text{enc}}(x)^\dagger U_{\text{var}}(\theta)^\dagger.$$

Finally, one measures a suitable observable $O(x, \theta)$ on this state. The scalar output is given by the trace or $f(\theta, x) = \text{tr}[O(x, \theta) \rho(x, \theta)]$, which is the expectation value of the measurement and serves as the prediction of the quantum model for input x and parameters θ .

Quantum Neural Networks

We view a quantum neural network as in (Bowles et al., 2024) as a model that uses one or more variational quantum circuits $f_1, \dots, f_L$ and combines their scalar outputs through a classical prediction function f_{pred} to produce a binary label $y \in \{\pm 1\}$:

$$y = f_{\text{pred}}(f_1(\theta_1, x), \dots, f_L(\theta_L, x)).$$

In the simplest case, we use only one quantum circuit ($L = 1$) and measure an operator whose possible outcomes are -1 or $+1$. The circuit output is then a number between -1 and $+1$ (its expectation value), and we choose the predicted class just by its sign: a positive value means class $+1$, while a negative value means class -1 .

Training proceeds analogously to classical neural networks. Given a dataset $\{(x_i, y_i)\}_{i=1}^N$, we define an average loss as

$$\mathcal{L}(\theta; X, y) = \frac{1}{N} \sum_{i=1}^N \ell(\theta, x_i, y_i),$$

where $\ell(\theta, x_i, y_i)$ is a pointwise loss and (X, y) collect all inputs and labels. The parameters $\theta = (\theta_1, \dots, \theta_L)$ are then optimised using stochastic gradient descent or related methods. Gradients with respect to quantum circuit parameters are obtained via the chain rule and parameter-shift rules (Mitarai et al., 2018; Schuld et al., 2019).

Quantum Kernel Methods

Quantum kernel methods adapt classical kernel-based learning (such as support vector machines) to the quantum setting by using a quantum device to define and evaluate the kernel function. The key idea is to embed each classical data point x into a quantum state $\rho(x)$ via a feature map (encoding circuit), and to define the kernel as an inner product between such states, for example

$$k(x_i, x_j) = \text{tr} [\rho(x_i) \rho(x_j)].$$

This quantum kernel can then be plugged into a standard classical kernel algorithm, which operates exactly as in the classical case but now uses similarities computed by a quantum circuit. In this way, the “model” remains a classical kernel method, while the quantum computer is used only to realise a potentially rich feature space that may be hard to simulate classically.

2.4 Motivation: The Benchmarking Problem in QML

Before large, fault-tolerant quantum computers are available, almost all empirical evidence about quantum machine learning (QML) comes from classical simulations of small quantum models or from experiments on noisy, few-qubit devices. In this regime, *benchmarking*, systematically comparing quantum models against each other and against classical baselines on common tasks, is one of the few tools available to assess whether current ideas in QML actually deliver on their promised advantages, as discussed in Bowles et al. (2024).

However, as Bowles et al. (2024) argue, drawing robust conclusions from such benchmarks is surprisingly difficult: small design choices in datasets, model architectures, metrics, and hyperparameter tuning can drastically change which model “wins”. This makes it hard to answer even seemingly simple questions such as whether a given quantum model is “better than” a classical one.

- **Dataset choice has a huge impact.** Classical “no free lunch” results already tell us that average performance of any learning algorithm depends strongly on the distribution of tasks considered. In practice, this means that simply changing which datasets are included in a benchmark, or how they are preprocessed, can completely reorder the ranking of methods. Empirically, even for standard classical models, fixing label errors in a dataset, changing the train–test split, or switching to a different evaluation metric can significantly shift which model appears best. In the QML setting, where many studies rely on only one or two small datasets, this sensitivity is even more problematic. Papers that illustrate this effect include (Dehghani et al., 2021; Northcutt et al., 2021; Recht et al., 2018, 2019).
- **Benchmark design embeds many hidden choices.** Beyond the data itself, numerous “small” design decisions shape the outcome:
 - Architectural tweaks (new ansätze, exotic layers, etc.) are often advertised as innovations, but large-scale benchmarks suggest that they frequently have little consistent effect on performance.
 - Hyperparameter optimisation (learning rates, layer counts, regularisation, kernel parameters, etc.) typically dominates performance: with enough tuning, a simple baseline can match or surpass more elaborate proposals.

- Different evaluation metrics (accuracy, F1-score, AUC, calibration measures, or cost-sensitive scores) reward different behaviour and may correlate poorly, so “best” can mean different things depending on what is measured.
- When computational cost is taken into account (e.g., training time, circuit evaluations, required hardware or simulator resources), rankings can reverse: a slightly more accurate model may be clearly inferior when efficiency is factored in.

For QML, these issues are amplified because each circuit evaluation is expensive and gradients can require thousands of shots or full-state simulations, so hyperparameter searches are hardware- and time-intensive. Papers that illustrate this effect include (Narang et al., 2021; Smith et al., 2023; Lucic et al., 2018; Henderson et al., 2017; Riquelme et al., 2018; Dodge et al., 2019).

- **Systematic positivity bias in the literature.** A simple thought experiment illustrates how publication practices may create a risk of positivity bias: suppose many research groups each try a large number of quantum models and datasets, but only submit papers when they find a configuration where the quantum model outperforms a classical baseline. Even if, on average, classical models tend to do better, the literature will still be dominated by positive results that claim “quantum beats classical”. This creates a misleading impression of steady progress, even if the underlying picture is much more mixed.

The conclusion is that broad questions of the form “Are quantum models better than classical ones?” cannot be answered by isolated experiments on a handful of small datasets. Instead, what can be meaningfully asked are *narrow*, well-specified questions: for example, how the performance of a particular family of quantum models changes as a controlled difficulty parameter increases, or how sensitive a given ansatz is to noise, or how a quantum kernel compares to a specific classical kernel on a particular class of tasks.

Carefully designed benchmarks must therefore emphasise controlled variation, reproducibility, and ablation-style comparisons over simple leaderboard rankings.

What matters in a comparison? Given these challenges, Bowles et al. (2024) argue that benchmarking should move beyond simplistic leaderboard questions and instead ask *structural* questions about model design. Rather than only asking whether a quantum model outperforms a classical baseline,

we should try to understand *which parts* of the model are crucial and which can be replaced by classically simulable components without loss of performance. In other words, benchmarking should be used as an *ablation tool* to probe the role of quantum resources.

Concretely, they propose constructing “dequantised” variants of QML models by systematically removing or restricting those ingredients that are specifically quantum, for example by

- removing entangling gates and using only local, single-qubit rotations (seen in the paper),
- restricting the gate set to Clifford transformations,
- replacing unitary evolutions by stochastic, or
- simulating circuits with tensor-network methods such as Matrix Product States (MPS) with low bond dimension.

By comparing the original model to these non-quantum or classically tractable variants on the *same* benchmarks, one can probe to what extent genuine quantum effects are responsible for any observed performance differences. When a dequantised variant matches the original model, this suggests that “quantumness” was not the decisive ingredient, at least on the tasks and scales considered.

Motivated by this perspective, the present work adopts a similar philosophy. Rather than taking the presence of quantum circuits as evidence of a quantum advantage, we use benchmarking as a tool to dissect our models. After introducing baseline variational quantum classifiers, we will *remove* its specifically quantum features and study how performance changes. In particular, we investigate variants following the dequantisation strategies suggested by Bowles et al. (2024). By comparing the original and dequantised models on controlled benchmarks, our goal is to understand not only *how well* they perform, but also *which aspects of their design appear most dependent on quantum resources*.

Chapter 3

Methodology

This chapter describes the experimental setup used to study the role of entanglement in two quantum model families: the Circuit-Centric Classifier (CCC) and the IQP kernel classifier. The main goal is not only to compare models by final accuracy, but to construct controlled ablations of each architecture and examine how performance changes as entanglement is reduced. To keep the comparison as fair as possible, all variants are evaluated within the same benchmarking framework, on the same datasets, and under the same training and hyperparameter-search procedure.

3.1 Experimental Setup

All experiments are carried out within the benchmarking framework of Bowles et al. (2024), which provides a common interface for datasets, model evaluation and hyperparameter search. In this thesis, I keep the overall benchmark structure of the paper and study two model families implemented in the same codebase: the Circuit-Centric Classifier introduced by Schuld et al. (2018) and the IQP kernel classifier from Havlíček et al. (2019). For both families, the goal is to compare the original model against entanglement-reduced variants under matched benchmark conditions.

The practical motivation for this setup is straightforward: if the original model and the modified variants are trained under identical conditions, then differences in performance can be interpreted more directly as arising from architectural changes, rather than from differences in preprocessing, tuning budget, or evaluation protocol. This is especially important in QML benchmarks, where small implementation choices can otherwise dominate the comparison (Bowles et al., 2024).

3.2 Datasets

This thesis uses the same benchmark family as the reference study of Bowles et al. (2024). The intention is therefore not to introduce new datasets, but to reuse the same controlled tasks so that the effect of dequantization can be studied in a setting that is already established in the literature.

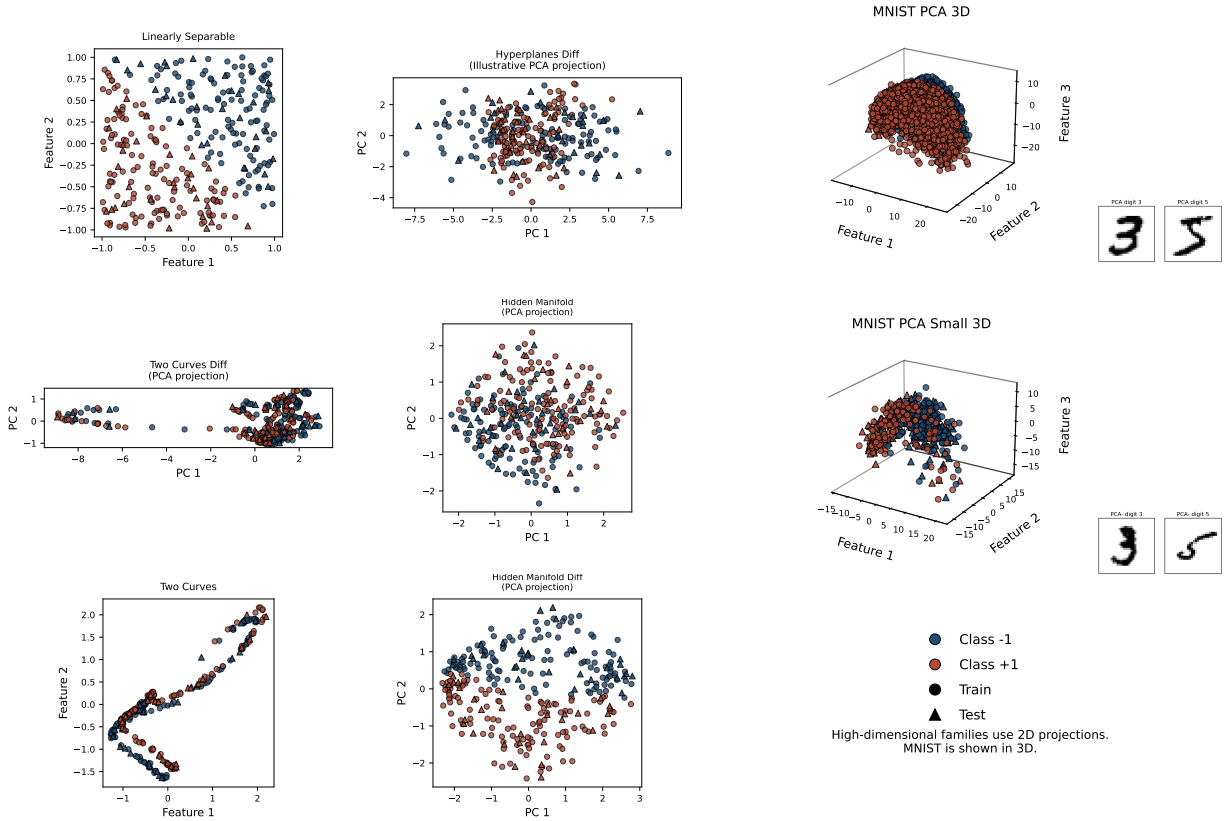


Figure 3.1: Illustrative examples of datasets created by the different data-generation procedures. For the scatter plots, the two classes are shown in blue and orange, and training points are shown as circles while test points are shown as triangles. High-dimensional benchmark families are displayed through two-dimensional PCA projections, while the MNIST families are shown in three dimensions together with example images of digits 3 and 5. The figure includes representative panels for the linearly separable, hyperplanes diff, hidden manifold, hidden manifold diff, two curves, two curves diff, MNIST PCA, and MNIST PCA small benchmarks.

The benchmark suite contains several dataset families, each designed to probe a different aspect of model behavior. Each benchmark vary either

the input dimension, the intrinsic structure of the data, or the geometric complexity of the decision boundary. This makes it possible to ask more specific questions than whether a model performs well on average.

At one end of the spectrum, the **linearly separable** benchmark serves as the simplest sanity-check task in the suite. The two classes are separated by a single hyperplane, so a basic linear classifier should already solve the problem. Its main role is therefore to test whether a model can recover a simple linear decision rule as the input dimension changes.

The **downscaled MNIST** benchmark provides a more realistic classification task while remaining tractable for QML simulations. It uses the handwritten-digit problem restricted to digits 3 versus 5, but in reduced form. This is performed as as low-dimensional PCA representations and as coarse-grained or lower resolution image representations for convolutional models. This dataset family is useful because it bridges the gap between highly controlled synthetic tasks and simplified real-image data.

The **hidden manifold** benchmarks are designed to mimic settings in which the observed data live or is in a high-dimensional space even though the true structure, or the manifold, is governed by fewer latent degrees of freedom, as is often the case in image and signal data. Two variants are particularly relevant. In **hidden manifold model**, the manifold dimension is held fixed while the visible input dimension increases. The underlying problem therefore remains the same, but it is embedded into a larger ambient space. In **hidden manifold diff**, by contrast, the visible dimension is fixed while the manifold dimension changes. Here the observed number of features stays constant, but the latent structure itself becomes more complex, making the task intrinsically harder.

The **two curves** benchmark tests nonlinear geometric classification by generating each class from a separate one-dimensional curve. These curves are embedded into a d -dimensional feature space. The curves are constructed using low-degree Fourier series, meaning combining sine and cosine functions, which makes smooth nonlinear structures, and Gaussian noise is added so points lie near the curves. The task is to classify a new point according to which of the two curves it most likely originated from. Two curves fixes the curve complexity and the offset between the curves, but increases the feature dimension d . Two curves diff fixes the feature dimension at $d=10$ but makes the curves more complex and closer together.

The **hyperplanes and parity** benchmark defines labels by combining several hyperplane tests. For each sample, the model must determine on which side of each hyperplane the point lies, and the final class is given by whether the number of positive-side outcomes is even or odd. This makes the task compositional, because the label depends on the combination of several

simple boundary relations rather than on a single separating hyperplane. In the **hyperplanes diff** variant, the task becomes harder by increasing the number of hyperplanes that must be combined.

The original benchmark suite of Bowles et al. (Bowles et al., 2024) also includes the **bars and stripes** dataset family, which consists of noisy images containing either vertical bars or horizontal stripes. However, this benchmark was not included in the present experiments due to its substantially higher computational cost in the considered simulation setting.

Taken together, these dataset families provide a controlled progression from simple linear tasks to problems with hidden low-dimensional structure, curved decision geometry, and compositional parity rules. It allows the role of entanglement to be studied across qualitatively different learning scenarios, rather than being inferred from a single benchmark alone.

3.3 Circuit-Centric Classifier

The first quantum model family studied in this thesis is the Circuit-Centric Classifier (CCC), following the architecture proposed by Schuld et al. (2018). Conceptually, CCC belongs to the variational, or quantum-neural-network, branch of QML discussed in Section 2.3. As such, it consists of trainable parameters arranged in layers and optimized from data to produce a classification score. However, instead of applying affine transformations and classical activation functions, CCC uses a parametrized quantum circuit acting on an encoded quantum state. In this sense, it follows the general framework of data embedding, variational ansätze, and measurement-based prediction introduced in the previous chapter.

3.3.1 Input Encoding

CCC uses amplitude encoding together with repeated copies of the encoded input state. In practice, if the classical feature dimension is not already a power of two, the input is padded accordingly. If d denotes the number of classical input features and C denotes the number of input copies, then each amplitude-encoded register requires

$$\lceil \log_2(d) \rceil$$

qubits. Therefore, the total number of qubits in the model is

$$M = C \cdot \lceil \log_2(d) \rceil.$$

This part of the model is inherited directly from the original CCC and is identical for the original and dequantized variants.

3.3.2 Variational Circuit and Prediction Rule

After the embedding, the original CCC pipeline uses PennyLane’s variational block **StronglyEntanglingLayers** (PennyLane, 2026b). This block consists of several consecutive layers. Each layer first applies trainable single-qubit rotations to all qubits, followed by entangling gates between pairs of qubits. Across layers, the entangling pattern changes by using larger ranges.

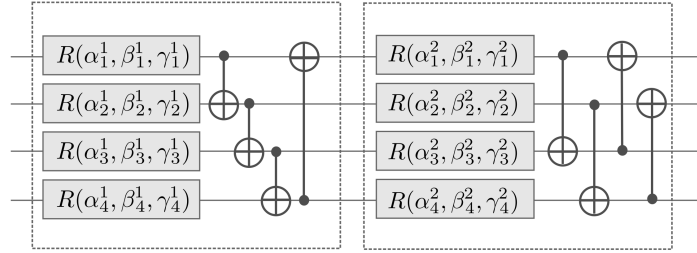


Figure 3.2: Example of two 4-qubit strongly entangling layers (ranges $r = 1$ and $r = 2$, respectively) with rotations R and CNOTs as imprimitives, adapted from PennyLane (2026b).

The model output afterwards is computed from the expectation value of the Pauli- Z observable measured on the first qubit. This expectation value is a scalar in the interval $[-1, 1]$, to which a trainable bias term b is added to obtain the final score. The prediction rule is kept fixed across all CCC variants and the only systematic modification concerns the internal entangling structure of the variational circuit.

3.4 Entanglement Ablation

We consider controlled modifications of CCC designed to reduce or remove entanglement while preserving the rest of the architecture as far as possible. This follows the ablation perspective advocated by Bowles et al. (2024).

To this end, I introduce dequantized CCC variants that keep the same embedding, the same measurement rule, the same training pipeline, and the same overall role of the variational block, but replace the original ansatz by a custom controlled construction in which the amount of entanglement can be adjusted directly.

3.4.1 Controlled Ansatz

The modified ansatz is built layer by layer. Each layer first applies a trainable general single-qubit rotation on every qubit,

$$\text{Rot}(\alpha, \beta, \gamma),$$

so the local trainable degrees of freedom are retained in all variants. The difference between variants lies only in how many of the candidate two-qubit couplings are actually activated.

Let M be the total number of qubits. In layer ℓ , the candidate entangling edges are defined by a cyclic offset

$$r_\ell = (\ell \bmod (M - 1)) + 1,$$

which produces the ring-like edge set

$$E_\ell = \{(i, i + r_\ell \bmod M) : i = 0, \dots, M - 1\}.$$

This means that successive layers cycle through different interaction ranges. From the candidate edge set, only a fraction is activated. If $\alpha \in [0, 1]$ denotes the entanglement-control parameter, then the number of active entangling gates in layer ℓ is

$$k_\ell = \text{round}(\alpha |E_\ell|).$$

Here, $\text{round}(\cdot)$ denotes rounding to the nearest integer. The active edges are chosen by seeded pseudorandom sampling, making the construction reproducible across runs while avoiding the same deterministic subset in every layer.

Operationally, α controls how much entangling structure is available to the circuit:

- $\alpha = 0$ removes all two-qubit gates and yields a fully separable circuit,
- $0 < \alpha < 1$ produces partially entangling circuits, and
- $\alpha = 1$ refers to all the entangling gates activated, which is equivalent to `StronglyEntanglingLayers`.

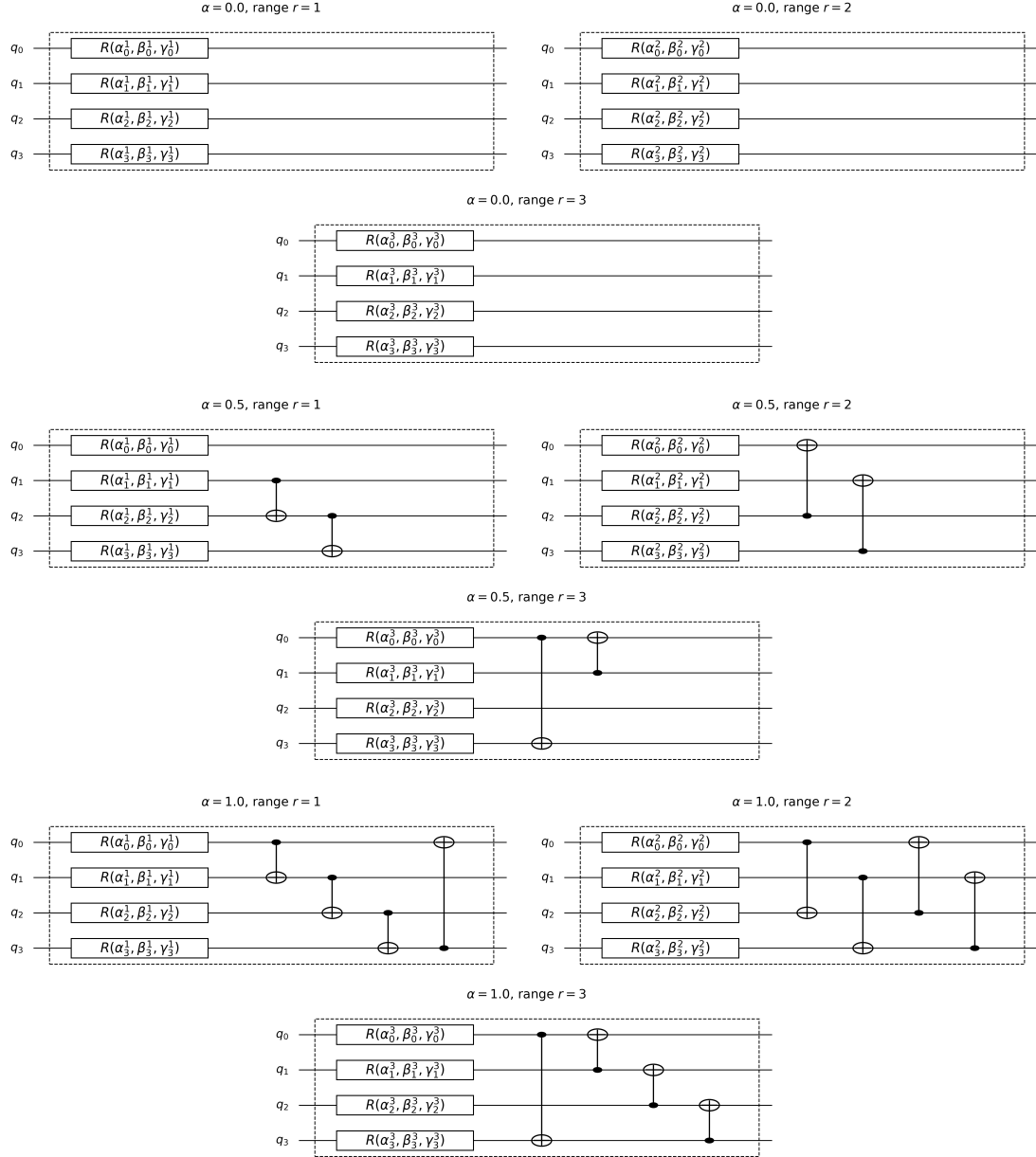


Figure 3.3: Example of the controlled CCC ansatz for $M = 4$ qubits and three interaction ranges $r = 1, 2, 3$. Each panel shows one layer consisting of local rotations $R(\alpha_i^\ell, \beta_i^\ell, \gamma_i^\ell)$, followed by the active CNOT couplings selected by the entanglement-control parameter.

3.4.2 CCC Variants

Two dequantized variants are considered in this work, namely, *CircuitCentricClassifierHalfSeparable* and *CircuitCentricClassifierSeparable*.

In the separable version, the entanglement-control parameter is set to $\alpha = 0$. As a result, no two-qubit gates are applied anywhere in the variational block. The circuit therefore remains fully local, and this constitutes the strongest dequantization step considered.

The *half-separable* has the entanglement-control parameter set to $\alpha = 0.5$, so that approximately half of the candidate two-qubit couplings in each layer remain active. This model occupies an intermediate position between the original entangling CCC and the fully separable version. It is methodologically useful because it allows us to test whether performance degrades gradually as entanglement is reduced, or whether most of the observed behavior can already be reproduced with only limited entangling structure. It uses a seeded random 50% subset instead, which preserves reproducibility.

The implementation of this controlled ansatz is shown in Listing 3.2, which is a modified version from 3.1.

```

1  for l in range(n_layers):
2
3      # Single-qubit rotation on every qubit
4      for i in range(len(wires)):
5          op_list.append(
6              Rot(
7                  weights[... , l, i, 0],
8                  weights[... , l, i, 1],
9                  weights[... , l, i, 2],
10                 wires=wires[i],
11             )
12         )
13
14     if len(wires) > 1:
15         # Candidate entangling edges for range ranges[l]
16         for i in range(len(wires)):
17             act_on = wires.subset(
18                 [i, i + ranges[l]],
19                 periodic_boundary=True,
20             )
21
22         # All candidate edges are active
23         op_list.append(imprimitive(wires=act_on))

```

Listing 3.1: Relevant logic of PennyLane’s `StronglyEntanglingLayers` template (PennyLane, 2026b).

```

1  for l in range(self.n_layers):
2
3      # Single-qubit rotation on every qubit
4      for i in range(M):
5          qml.Rot(
6              weights[l, i, 0],

```



```

7         weights[l, i, 1],
8         weights[l, i, 2],
9         wires=i,
10    )
11
12    if M > 1:
13        # Interaction range for layer l
14        r = (l % (M - 1)) + 1
15
16        # Ring-like candidate entangling edge set
17        edges = [(i, (i + r) % M) for i in range(M)]
18
19        # Entanglement-control parameter alpha
20        # alpha = self.entanglement_level
21        k = int(round(self.entanglement_level * len(edges)))
22
23        if k > 0:
24            rng = np.random.default_rng(
25                self.random_state + 1000 * l
26            )
27
28            # Active entangling edges
29            idx = rng.choice(len(edges), size=k, replace=False)
30            idx.sort()
31
32            for j in idx:
33                a, b = edges[j]
34                gate(wires=[a, b])

```

Listing 3.2: Relevant controlled-ansatz logic implemented in `src/qml_benchmarks/models/circuit_centric.py`.

The concrete half-separable and fully separable CCC variants are obtained through wrapper classes that use the same controlled ansatz but fix `entanglement_level` to 0.5 and 0.0, respectively, as shown below.

<pre> class HalfSeparable(CircuitCentricClassifier): def __init__(self, ...): super().__init__(ansatz_type="controlled", entanglement_level=0.5, ...) </pre>	<pre> class Separable(CircuitCentricClassifier): def __init__(self, ...): super().__init__(ansatz_type="controlled", entanglement_level=0.0, ...) </pre>
---	---

Listing 3.3: Wrappers for half-separable and fully separable CCC variants.

3.5 IQP Kernel Classifier

The second quantum model family is the IQP kernel classifier from Havlíček et al. (2019), implemented here through the `IQPKernelClassifier` already present in the benchmark codebase. This family is included so that the entanglement-ablation question can be asked not only for a variational classifier, but also for a quantum-kernel model. In contrast to CCC, the IQP

kernel model does not learn circuit parameters directly. Instead, it replaces the classical similarity function used in kernel methods, discussed in Section 2.2.3, by one induced through a quantum feature map.

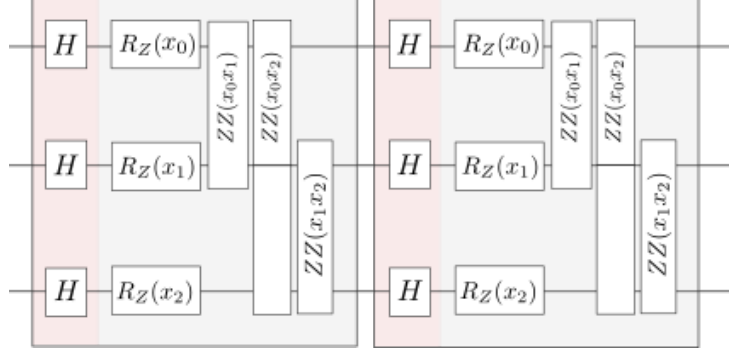


Figure 3.4: Example of an IQP embedding circuit with three input features and two repeated embedding blocks. Adapted from PennyLane (PennyLane, 2026a).

The feature map follows the logic of PennyLane’s `qml.IQPEmbedding` template (PennyLane, 2026a). In each repeat, the template applies a layer of Hadamard gates and single-qubit R_Z rotations that encode the individual features, followed by ZZ interactions that encode pairwise feature products. The resulting circuit defines a kernel of the form

$$k(x, x') = |\langle 0 | U(x')^\dagger U(x) | 0 \rangle|^2,$$

which is then passed to a classical support-vector machine with a precomputed kernel matrix. In other words, the quantum circuit defines the feature space, while the final classifier remains a classical SVM.

3.6 Entanglement Ablation

For the IQP family, the ablation modifies the entangling structure inside this embedding while keeping the single-qubit encoding fixed. More specifically, all variants retain the same Hadamard and R_Z feature-encoding layers, and differ only in the ZZ entangler pattern. This is the natural quantity to vary because PennyLane’s `qml.IQPEmbedding` already allows the entangling wire pairs to be controlled through its `pattern` argument (PennyLane, 2026a). The hyperparameter interface is also kept aligned across variants, in particular with respect to the number of IQP repeats and the SVM regularisation parameter.

3.6.1 IQP Kernel Variants

The first variant is the original `IQPKernelClassifier`, which uses the full IQP-style entangler pattern in every repeat of the embedding.

The second is `IQPKernelClassifierSeparable`. This variant keeps the same single-qubit IQP encoding but removes all ZZ couplings. It is therefore the direct no-entanglement ablation of the IQP embedding.

The third is `IQPKernelClassifierHalfSeparable`. This variant again keeps the same single-qubit IQP encoding, but in each repeat retains only a seeded random 50% subset of the full ZZ edge set. As in the CCC case, the intention is to test whether performance degrades gradually as entangler density is reduced, or whether much of the kernel’s behavior can already be recovered with a sparse entangling pattern.

These IQP-kernel variants are methodologically useful because they allow the entanglement question to be posed in a different QML model class. In CCC, entanglement appears inside a trainable variational ansatz. In the IQP-kernel model, the entangling ZZ structure is part of the fixed feature map itself. Comparing both families therefore helps separate claims about entanglement in a particular ansatz from claims about entanglement in a broader QML pipeline.

Listing 3.4 gives the corresponding implementation in `src/qml_benchmarks/models/iqp_kernel.py`. It uses the relevant logic of PennyLane’s `qml.IQPEmbedding` template (PennyLane, 2026a) in `_apply_iqp_embedding`. Listing 3.5 then shows the variant-specific pattern overrides.

```
1 def _full_iqp_entangler_pattern(wires):
2     # Full pairwise ZZ entangler pattern
3     return tuple(combinations(wires, 2))
4
5
6 class IQPKernelClassifier(ClassifierMixin, BaseEstimator):
7     def _repeat_entangler_patterns(self, wires):
8         # Use the same full pattern in every repeat
9         full_pattern = _full_iqp_entangler_pattern(wires)
10        return tuple(full_pattern for _ in range(self.repeats))
11
12    def _apply_iqp_embedding(self, features, wires, repeat_patterns):
13        for pattern in repeat_patterns:
14            # As in qml.IQPEmbedding: Hadamards and RZ feature encoding
15            for idx, wire in enumerate(wires):
16                qml.Hadamard(wires=wire)
17                qml.RZ(features[idx], wires=wire)
18
19            # As in qml.IQPEmbedding: ZZ entanglers on the chosen pattern
20            for wire_pair in pattern:
21                idx_a = wires.index(wire_pair[0])
22                idx_b = wires.index(wire_pair[1])
23                angle = features[idx_a] * features[idx_b]
24                qml.MultiRZ(angle, wires=wire_pair)
```

Listing 3.4: Relevant IQP-kernel logic implemented in

src/qml_benchmarks/models/iqp_kernel.py.

```
1 def _random_half_iqp_patterns(wires, n_repeats, random_state):
2     full_pattern = _full_iqp_entangler_pattern(wires)
3
4     if len(full_pattern) <= 1:
5         return tuple(tuple(full_pattern) for _ in range(n_repeats))
6
7     # Keep about half of the full ZZ edge set in each repeat
8     target_n_edges = max(1, len(full_pattern) // 2)
9     patterns = []
10
11     for repeat in range(n_repeats):
12         rng = np.random.default_rng(random_state + 1000 * repeat)
13         idx = rng.choice(len(full_pattern), target_n_edges, replace=False)
14         idx = np.sort(idx)
15         patterns.append(tuple(full_pattern[i] for i in idx))
16
17     return tuple(patterns)
18
19
20 class IQPKernelClassifierSeparable(IQPKernelClassifier):
21     def _repeat_entangler_patterns(self, wires):
22         # Remove all ZZ couplings in every repeat
23         return tuple(tuple() for _ in range(self.repeats))
24
25
26 class IQPKernelClassifierHalfSeparable(IQPKernelClassifier):
27     def _repeat_entangler_patterns(self, wires):
28         # Keep a seeded random 50% subset of the full pattern
29         return _random_half_iqp_patterns(
30             wires, self.repeats, self.random_state
31         )
```

Listing 3.5: Variant-specific entangler-pattern logic for the separable and half-separable IQP-kernel classifiers.

3.7 Training Procedure

All model variants are trained within the same benchmark pipeline. This includes the same dataset generation, the same train-test protocol, the same hyperparameter-search infrastructure, and the same result logging format. The purpose of this uniform setup is to ensure that the comparison between CCC and its dequantized variants is an ablation study of model structure rather than a comparison between different experimental conditions.

For CCC, the implementations are trained as binary classifiers with labels in $\{-1, +1\}$. The model parameters consist of the variational circuit parameters together with the scalar bias term. Hyperparameters such as the number of input copies, the number of variational layers and learning rate are selected through the same search procedure used by the benchmark frame-

work. This point matters because, as emphasized by Bowles et al. (2024), hyperparameter tuning can strongly affect benchmark outcomes in QML.

For the IQP-kernel family, the training flow is different but the benchmarking philosophy is the same. The quantum circuit is used to construct kernel matrices, and classification is then performed by a classical SVM backend. The main hyperparameters are the number of IQP repeats and the SVM regularisation parameter.

3.8 Evaluation

Model performance is evaluated primarily through test accuracy on the benchmark tasks. The central object of interest is the relative ranking of the original and dequantized CCC variants and IQP Kernel variants across datasets and benchmark settings. The experiments therefore examine not only absolute scores, but also whether removing entanglement changes the ranking of models across tasks of different difficulty.

All reported results come from the benchmark framework described in this chapter. Train accuracy is used only as a secondary diagnostic to assess whether changes in test performance are associated with changes in fitting capacity or with possible overfitting.

3.9 Reproducibility

All experiments were conducted within a modified benchmark setup based on the `qml_benchmarks` repository. The exact codebase used in this thesis is available at <https://github.com/Mauricio-GutierrezL/Thesis>. This repository provided the common experimental pipeline for dataset handling, model training, hyperparameter selection, and final evaluation. The overall benchmarking methodology follows the framework discussed by Bowles et al. (2024). The computational environment was based on Python 3.9.21 and used PennyLane 0.38.0 together with JAX, with 64-bit precision enabled via `jax.config.update("jax_enable_x64", True)`. Additional core dependencies included scikit-learn, NumPy, and matplotlib. The modified implementations of the Circuit-Centric Classifier variants introduced in this thesis are located in `src/qml_benchmarks/models/circuit_centric.py`, while the IQP-kernel variants are implemented in `src/qml_benchmarks/models/iqp_kernel.py`. Benchmark datasets were stored under `thesis/datasets_tests/`, while experiment outputs and aggregated results were written to `thesis/my_results/`.

All runs were executed on the RWTH Aachen CLAIX HPC cluster using CPU-only jobs scheduled through Slurm. Hyperparameter optimization was performed with `GridSearchCV` using five-fold cross-validation, and the same train/test evaluation pipeline was applied uniformly across all compared models in order to ensure methodological consistency.

Benchmark datasets were generated with the utility script `thesis/generate_paper_benchmarks.py`, built on top of the data-generation functions in `src/qml_benchmarks/data/`. The benchmark-specific parameter choices in this script follow the original benchmark-generation setup from `paper/benchmarks/`. Dataset generation used fixed seeds per benchmark family, including 42 for linearly separable and MNIST-PCA data, 3 for hidden-manifold and two-curves families, and 1 for hyperplanes diff.

Model-level randomness was controlled through the classifier parameter `random_state`. In the controlled CCC ansatz, this parameter determines the seeded pseudorandom selection of active entangling edges, ensuring reproducibility of the selected edge subsets. In the IQP-kernel family, `random_state` initializes the model RNG, from which JAX PRNG keys and the SVM backend’s random seed are derived. For `IQPKernelClassifierHalfSeparable`, it also determines the seeded random 50% subset of ZZ entangling edges selected in each IQP repeat through `_random_half_iqp_patterns`. During final scoring, each selected model was evaluated over five runs with `random_state` values $0, \dots, 4$. In addition, the execution scripts `scripts/run_hyperparameter_search.py` and `scripts/score_with_best_hyperparameters.py` set `np.random.seed(42)` globally in order to make script-level NumPy randomness reproducible.

Chapter 4

Results

4.1 Quantitative Results

Figure 4.1 summarizes the aggregate ranking behavior of the three CCC variants across the completed benchmark set. For each model, the stacked bar shows how often it finished in each rank across all datasets on which it was tested, with blue indicating first place, white indicating second place, and red indicating last place. The models are ordered from top to bottom by average normalized rank, so better-performing models appear higher in the figure. Figure 4.3 summarizes it as well but per benchmark family.

The original entangling CCC is the strongest model overall. It achieves the best test accuracy on 90 instances, while the half-separable variant is best on 31 instances and the fully separable model is best on 13 instances. This shows a clear hierarchy: the original model is usually strongest, the half-separable variant is often competitive, and the fully separable model is only occasionally best.

Relative to the original CCC, the half-separable variant shows only a small mean loss in test accuracy across the completed benchmark set, about -0.0116 . By contrast, the fully separable model is markedly worse, with a mean test-accuracy difference of about -0.1705 relative to the original CCC. This can be seen in the first row of Figure 4.2. The difference between the half-separable and fully separable variants is then correspondingly large, about $+0.1590$ in favor of the half-separable model. The main empirical pattern is therefore not that every reduction of entanglement is equally harmful, but that removing entanglement entirely is much more damaging than thinning it out.

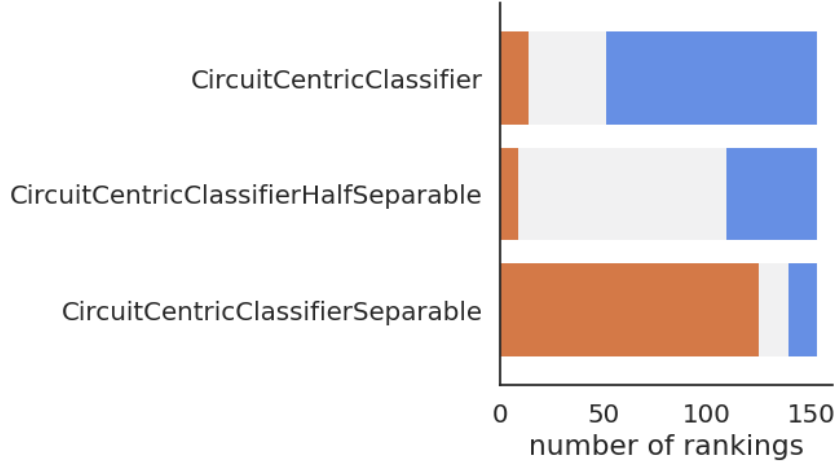


Figure 4.1: Number of rankings (blue/first to red/last) across all datasets on which each model was tested, for the three CCC model families. The models are ordered from top to bottom by average normalized rank.

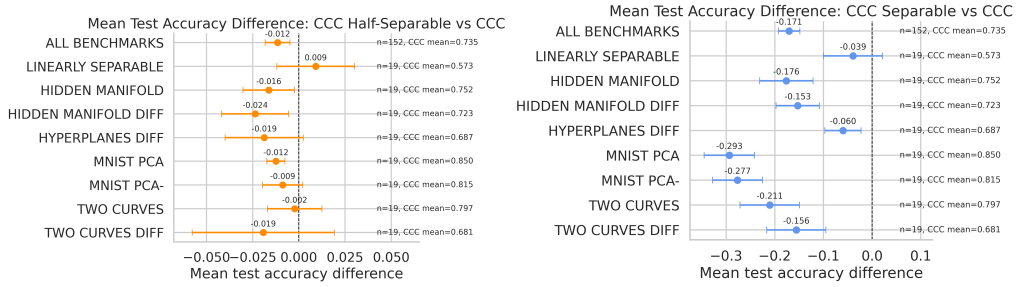


Figure 4.2: Forest plots of paired mean test-accuracy differences for the dequantized CCC variants relative to the original CCC. Each point is the average benchmark-wise test-accuracy difference within a family, and each horizontal interval shows the corresponding 95% confidence interval across matched benchmark instances. Negative values indicate worse performance than the original CCC. The horizontal scales differ between the two panels.

This pattern is also visible when the results are broken down by benchmark family, as shown by the forest plots in Figure 4.2. On linearly separable data, the half-separable variant is slightly better on average than the original model, by about $+0.0091$, while the fully separable model is slightly worse, by about -0.0391 . This suggests that for the simplest tasks in the completed benchmark set, the full entangling structure may not be necessary. On hyperplanes diff, the three variants are comparatively close, with the separable model behind the original by only about -0.0598 . By contrast,

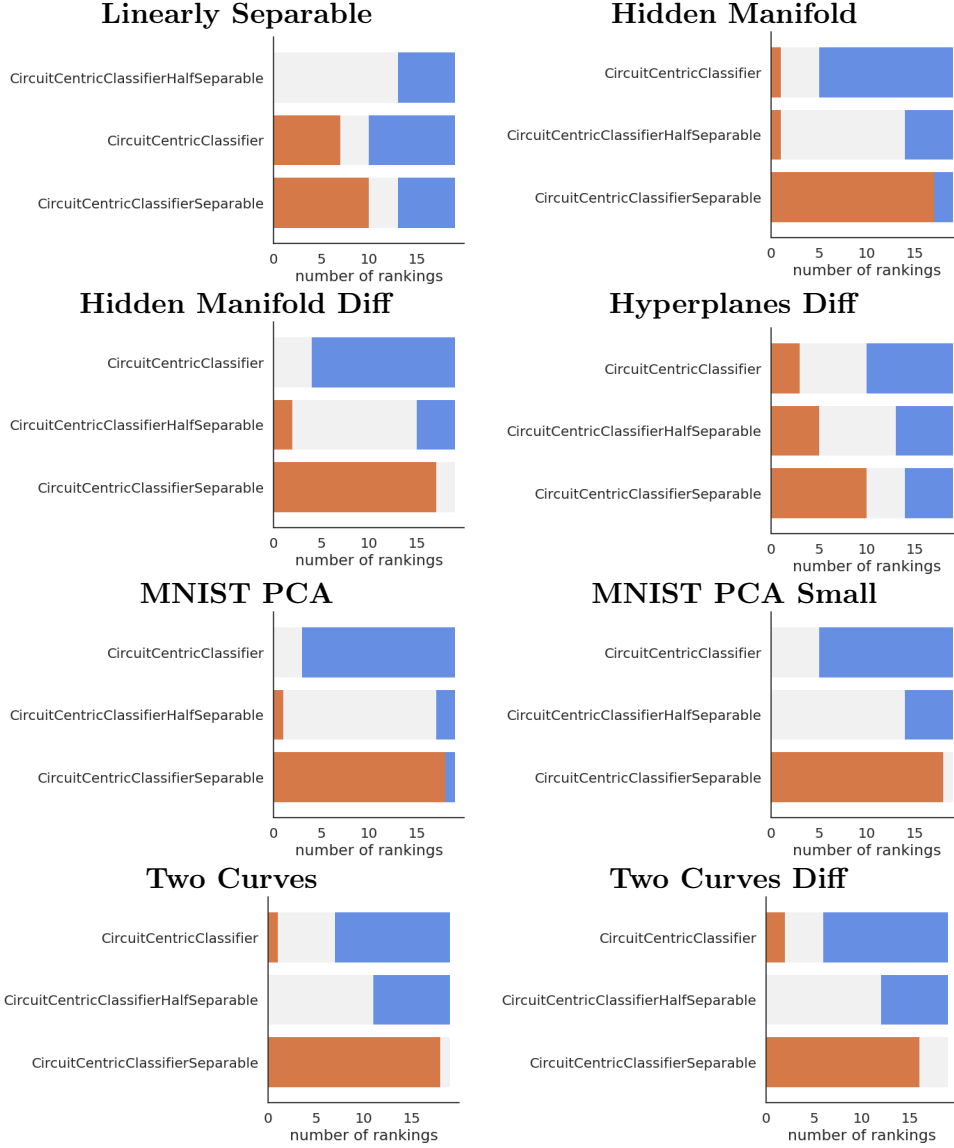


Figure 4.3: Per-family ranking plots for all completed benchmark families: linearly separable, hidden manifold, hidden manifold diff, hyperplanes diff, MNIST PCA, MNIST PCA small, two curves, and two curves diff.

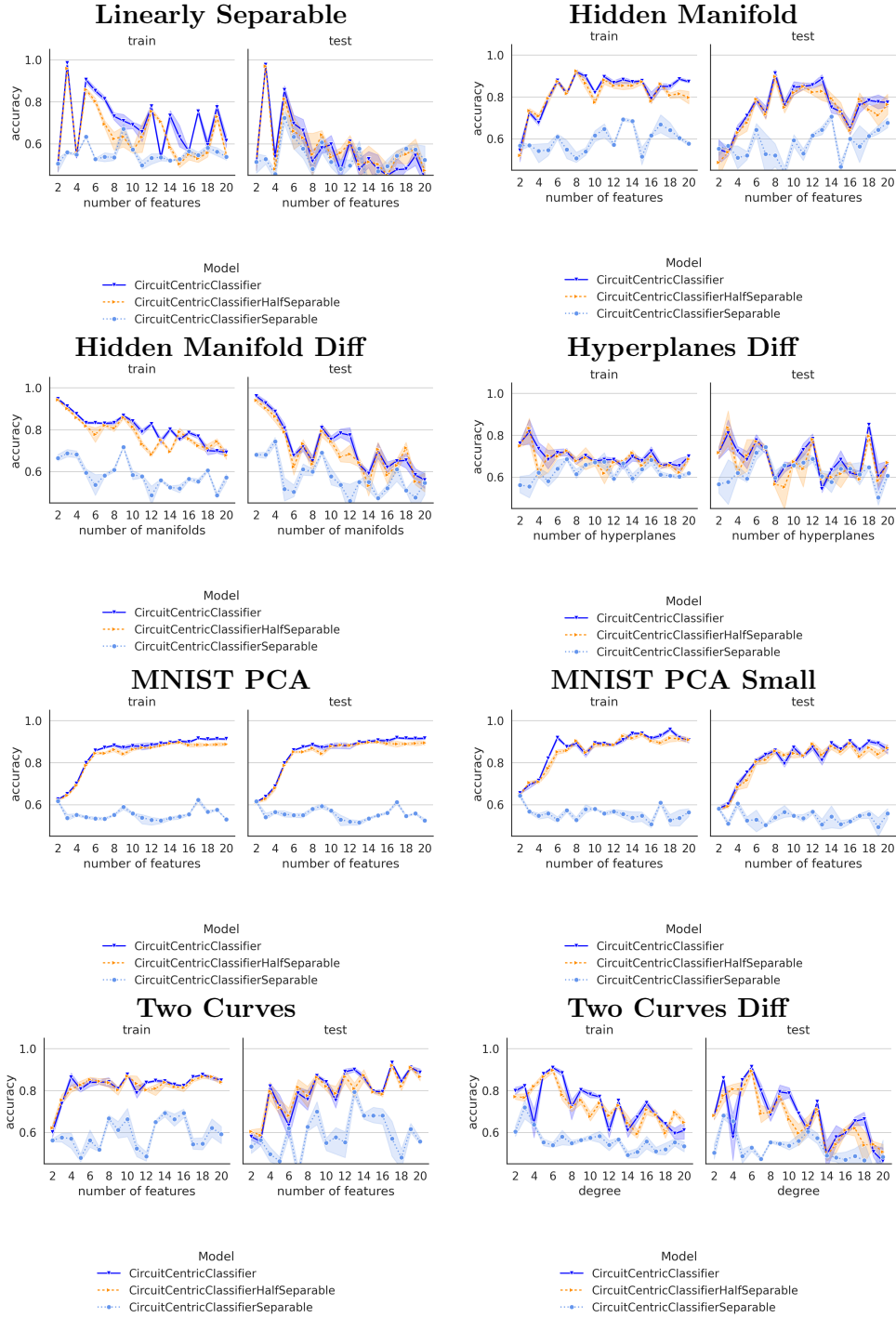


Figure 4.4: Train and test accuracy curves for all completed benchmark families: linearly separable, hidden manifold, hidden manifold diff, hyperplanes diff, MNIST PCA, MNIST PCA small, two curves, and two curves diff.

on hidden manifold, hidden manifold diff, MNIST PCA, MNIST PCA small, and two curves diff, the fully separable model is consistently and often substantially worse than the original CCC. The effect is especially pronounced on the MNIST PCA families, where the separable model trails the original by about -0.2934 and -0.2765 , respectively.

The half-separable variant occupies an intermediate position. On most benchmark families it is slightly worse than the original CCC, but the drop is small compared with the fully separable case. For example, the average test-accuracy difference half-minus-full is about -0.0163 on hidden manifold, -0.0237 on hidden manifold diff, -0.0188 on hyperplanes diff, -0.0124 on MNIST PCA, -0.0088 on MNIST PCA small, and -0.0193 on two curves diff. These values are modest enough to suggest that, across the completed CCC benchmark families, a sparse entangling structure retains a substantial fraction of the original model’s performance, which is also reflected in the train and test curves shown in Figure 4.4.

The train-versus-test comparison is broadly consistent with this interpretation. Averaged over the completed benchmark set, the original CCC reaches about 0.7886 train accuracy and 0.7347 test accuracy, while the half-separable variant reaches about 0.7698 train accuracy and 0.7232 test accuracy. The separable model is much lower on both metrics, with about 0.5775 train accuracy and 0.5642 test accuracy. In other words, the separable model does not merely generalize worse after fitting the data equally well; it appears to have substantially lower effective fitting capacity on these tasks. This can be seen in Table 4.1.

Benchmark	CCC			Half-Sep			Sep		
	Train	Test	Gap	Train	Test	Gap	Train	Test	Gap
Linearly Separable	0.703	0.573	0.130	0.647	0.582	0.064	0.553	0.534	0.018
Hidden Manifold	0.827	0.752	0.075	0.810	0.735	0.075	0.593	0.575	0.017
Hidden Manifold Diff	0.807	0.723	0.084	0.782	0.699	0.083	0.584	0.570	0.014
Hyperplanes Diff	0.699	0.687	0.012	0.686	0.668	0.017	0.625	0.627	-0.002
MNIST PCA	0.849	0.850	-0.000	0.834	0.837	-0.003	0.554	0.556	-0.002
MNIST PCA Small	0.868	0.815	0.053	0.856	0.807	0.050	0.559	0.539	0.020
Two Curves	0.823	0.797	0.026	0.819	0.794	0.024	0.590	0.586	0.004
Two Curves Diff	0.733	0.681	0.051	0.725	0.662	0.063	0.562	0.526	0.036

Table 4.1: Family-level averages of train accuracy, test accuracy, and train-test gap for the three CCC variants across the completed benchmark set. Darker blue indicates higher accuracy; darker orange indicates a larger train-test gap. The gap is defined as train minus test.

The main quantitative conclusion from the completed CCC data is the

following: the original entangling CCC is usually best, but the half-separable variant remains competitive, whereas the fully separable model often incurs a substantial loss. Entanglement appears to matter in this benchmark setting, but the completed results suggest that full entangler density is often not required to obtain most of the performance of the original model on the tasks studied here.

4.2 Qualitative Interpretation

The dataset families probe different kinds of structure, and the effect of entanglement is not uniform across them.

The linearly separable benchmark primarily tests whether a model can recover a simple hyperplane-based decision rule; a task a simple linear classifier should be able to do. Since this structure does not require complex nonlinear or compositional feature interactions, small differences between the CCC variants on this family are consistent with the expectation that strong entangling connectivity is not central for this task. However, the relatively modest absolute accuracies limit the strength of this interpretation. In particular, the slight advantage of the half-separable variant over the original CCC should not be over-interpreted as evidence against the usefulness of entanglement more broadly. The task may be simple enough that the full entangling structure provides more representational capacity than necessary, potentially leading to overfitting. This interpretation is supported by the larger train–test gap observed for the original CCC, which is the largest among all.

The hidden-manifold families tell a different story. There the fully separable model drops substantially, while the original and half-separable variants remain much closer. Since these benchmarks are intended to probe structure that is embedded nontrivially in a larger feature space; both as the hidden structure complexity remains constant and the feature dimensions grow in number and vice versa, the found pattern is consistent with the idea that some nonlocal interaction structure in the variational block helps the model align with the task.

Similarly, MNIST PCA families show a particularly pronounced degradation of the fully separable model relative to the original CCC and the half-separable variant. Although the separable model also performs poorly on linearly separable and two-curves-diff benchmarks, those families are challenging for the entangling variants as well. The MNIST PCA case is therefore more distinctive, because the performance loss appears especially specific to the fully separable architecture. One possible explanation is that MNIST

PCA retains distributed image-like structure, where class-relevant information is spread across several input components. Without entangling operations, this information cannot be effectively mixed across qubits before the final first-qubit measurement, making the separable variant particularly susceptible to an architectural bottleneck.

The standard two-curves benchmark shows a qualitatively similar pattern to the MNIST PCA families: removing entanglement entirely leads to a pronounced degradation of the fully separable model, while the half-separable variant remains much closer to the original CCC. The two-curves-diff benchmark, however, should be interpreted separately. Its lower performance for the original CCC and the half-separable variant can be attributed to the construction of the task itself. While the standard two-curves benchmark keeps the curve complexity fixed, two-curves-diff increases the geometric complexity of the underlying curves at fixed feature dimension. The resulting decision structure is more intricate and may require the model to capture more oscillatory nonlinear relations between features. Therefore, the reduced performance of the entangling variants on two-curves-diff is better understood as reflecting the intrinsic difficulty of the benchmark, rather than only the effect of entanglement reduction.

The results for the hyperplanes diff benchmark show a somewhat different pattern from the other benchmark families. For the CCC and HalfSeparable models, which both retain entangling operations to different degrees, hyperplanes diff performs relatively poorly compared with the other benchmarks. In contrast, the fully separable model performs comparatively well on this benchmark. One possible explanation is that this task depends less on learning rich entangled feature interactions and more on combining several local boundary relations. In this setting, the local rotations of the separable model may still be sufficient to capture a substantial part of the repeated hyperplane structure. To provide a contrast example, in hidden manifold diff, the hidden structure governing the data becomes richer and more complex, and in two curves diff, the Fourier degree increases, making the curves more oscillatory and geometrically intricate. Therefore, the classifier may need to integrate information across multiple dimensions or regions simultaneously. These properties would be expected to suffer more strongly when entanglement is removed.

Furthermore, although a small train-test gap can indicate good generalization, low train and test accuracies together more commonly suggest underfitting. This behavior appears repeatedly for the fully separable variant. This could be caused by the architectural limitation of the separable variant for which the measurement procedure remains on only the first qubit. Without entangling operations, information from distant qubits may therefore

influence the output less effectively, introducing an architectural bottleneck. Overall, the train-test evidence suggests that removing entanglement entirely weakens the model primarily by reducing its ability to fit the tasks, whereas partial entanglement preserves much of that fitting capacity.

4.3 Limitations

The results of this thesis should be interpreted within several important limitations.

First, all experiments were performed in a simulation-based setting. The reported results therefore characterize the behavior of the implemented model architectures under idealized simulator conditions, but they do not account for hardware-specific effects such as gate noise, finite-shot sampling, decoherence, calibration errors, or restricted device connectivity. Consequently, the experiments should not be interpreted as direct evidence of performance on present-day quantum hardware.

A second limitation concerns the type of data considered. The benchmark families used in this thesis consist of classical datasets that are embedded into quantum circuits through encoding procedures such as amplitude encoding or feature maps. This differs from quantum-dataset settings, where the data may consist directly of quantum states generated by quantum processes (Schatzki et al., 2021) or, more broadly, data generated by or characterizing quantum systems, such as state vectors, Hamiltonians, unitaries, measurement distributions, control pulses, and noise information (Perrier et al., 2021). The present work therefore studies quantum models applied to classical learning problems, rather than learning directly from intrinsically quantum-mechanical data obtained from physical or simulated quantum systems.

The scope of the model comparison is also limited. The main empirical analysis focuses on the Circuit-Centric Classifier and its entanglement-reduced variants, with only limited results available for the IQP kernel family. The conclusions should therefore not be understood as statements about quantum machine learning models in general. They apply specifically to the evaluated model families, benchmark tasks, and implementation choices. Similarly, the entanglement reduction studied here follows one particular controlled ablation strategy based on removing or sparsifying two-qubit couplings. Other dequantization strategies, different ansatz designs, or alternative measurement rules could lead to different behavior.

A further architectural limitation is specific to the CCC model. The prediction rule measures only the Pauli- Z expectation value of the first qubit.

In the original and partially entangling variants, entangling gates allow information from other qubits to influence this measured output qubit. In the fully separable variant, this information transfer is removed. As a result, part of the poor performance of the separable CCC may reflect a measurement bottleneck rather than only the absence of entanglement itself. Modifying the measurement rule could mitigate this issue, but it would constitute a more invasive architectural change than the entanglement ablation considered in this thesis.

The benchmark coverage is also constrained by computational resources, which were directly tied to the finite compute-time allocation available for the RWTH Aachen HPC project used in this thesis. Since the project quota limited the total number of CPU core-hours available for dataset generation, hyperparameter searches, and repeated scoring runs, not all planned experiments could be completed at the same scale. This particularly affected some benchmark families from the reference suite, such as bars-and-stripes, and the IQP kernel experiments. As for which quantum kernel evaluation requires constructing pairwise kernel matrices and therefore scales unfavorably with dataset size and hyperparameter settings.

Finally, the results remain subject to the usual limitations of empirical benchmarking. Performance can depend on the selected hyperparameter search space, optimization settings, random seeds, and evaluation metric. For this matter, selection followed (Bowles et al., 2024). Although repeated runs and fixed seeds were used to improve reproducibility, the analysis is still based on a finite number of experiments. Therefore, statistical variance is expected even with averaging.

Chapter 5

Conclusion and Outlook

Bibliography

- Benedetti, M., Lloyd, E., Sack, S., and Fiorentini, M. (2019). Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001.
- Bohr, N. (1913). On the constitution of atoms and molecules. *Philosophical Magazine*, 26:1–25.
- Born, M. (1926). Zur quantenmechanik der stoßvorgänge. *Zeitschrift für Physik*, 37:863–867.
- Bowles, J., Ahmed, S., and Schuld, M. (2024). Better than classical? The subtle art of benchmarking quantum machine learning models. *arXiv preprint arXiv:2403.07059*.
- Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.
- de Broglie, L. (1924). *Recherches sur la théorie des quanta*. Phd thesis, Université de Paris.
- Dehghani, M., Tay, Y., Gritsenko, A. A., Zhao, Z., Houlsby, N., Diaz, F., Metzler, D., and Vinyals, O. (2021). The benchmark lottery. *arXiv preprint arXiv:2107.07002*. arXiv preprint arXiv:2107.07002.
- Dodge, J., Gururangan, S., Card, D., Schwartz, R., and Smith, N. A. (2019). Show your work: Improved reporting of experimental results. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics.
- Einstein, A. (1905). Über einen die erzeugung und verwandlung des liches betreffenden heuristischen gesichtspunkt. *Annalen der Physik*, 17:132–148.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

- Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing (STOC 1996)*, pages 212–219. ACM.
- Havlíček, V., Córcoles, A. D., Temme, K., Harrow, A. W., Kandala, A., Chow, J. M., and Gambetta, J. M. (2019). Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212.
- Heisenberg, W. (1925). Über quantentheoretische umdeutung kinematischer und mechanischer beziehungen. *Zeitschrift für Physik*, 33:879–893.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D., and Meger, D. (2017). Deep reinforcement learning that matters. *CoRR*, abs/1709.06560. arXiv preprint arXiv:1709.06560.
- Lucic, M., Kurach, K., Michalski, M., Gelly, S., and Bousquet, O. (2018). Are GANs created equal? a large-scale study. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*.
- Mitarai, K., Negoro, M., Kitagawa, M., and Fujii, K. (2018). Quantum circuit learning. *Physical Review A*, 98(3):032309.
- Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press, Cambridge, MA.
- Narang, S., Chung, H. W., Tay, Y., Fedus, L., Fevry, T., Matena, M., Malkan, K., Fiedel, N., Shazeer, N., Lan, Z., Zhou, Y., Li, W., Ding, N., Marcus, J., Roberts, A., and Raffel, C. (2021). Do transformer modifications transfer across implementations and applications? In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5758–5773, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Nielsen, M. A. and Chuang, I. L. (2010). *Quantum Computation and Quantum Information*. Cambridge University Press. 10th Anniversary Edition.
- Northcutt, C. G., Athalye, A., and Mueller, J. W. (2021). Pervasive label errors in test sets destabilize machine learning benchmarks. *arXiv preprint arXiv:2103.14749*. arXiv preprint arXiv:2103.14749.
- PennyLane (2026a). qml.IQPEmbedding. <https://docs.pennylane.ai/en/stable/code/api/pennylane.IQPEmbedding.html>. Accessed: 2026-05-12.

- PennyLane (2026b). `qml.StronglyEntanglingLayers`. <https://docs.pennylane.ai/en/stable/code/api/pennylane.StronglyEntanglingLayers.html>. Accessed: 2026-05-11.
- Perrier, E., Youssry, A., and Ferrie, C. (2021). Qdataset: Quantum datasets for machine learning. *arXiv preprint arXiv:2108.06661*.
- Planck, M. (1900). Zur theorie des gesetzes der energieverteilung im normalspektrum. *Verhandlungen der Deutschen Physikalischen Gesellschaft*, 2:237–245.
- Rath, M. and Date, H. (2023). Quantum data encoding: A comparative analysis of classical-to-quantum mapping techniques and their impact on machine learning accuracy. *arXiv preprint arXiv:2311.10104*. Department of Decision Science, IIM Mumbai.
- Recht, B., Roelofs, R., Schmidt, L., and Shankar, V. (2018). Do cifar-10 classifiers generalize to cifar-10? *arXiv preprint arXiv:1806.00451*. arXiv preprint arXiv:1806.00451.
- Recht, B., Roelofs, R., Schmidt, L., and Shankar, V. (2019). Do imagenet classifiers generalize to imagenet? In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400. PMLR.
- Riquelme, C., Tucker, G., and Snoek, J. (2018). Deep bayesian bandits showdown: An empirical comparison of bayesian deep networks for thompson sampling. *arXiv preprint arXiv:1802.09127*. arXiv preprint arXiv:1802.09127.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.
- Schatzki, L., Arrasmith, A., Coles, P. J., and Cerezo, M. (2021). Entangled datasets for quantum machine learning. *arXiv preprint arXiv:2109.03400*.
- Schölkopf, B. and Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.
- Schrödinger, E. (1926). Quantisierung als eigenwertproblem. *Annalen der Physik*, 384:361–376.
- Schuld, M., Bergholm, V., Gogolin, C., Izaac, J., and Killoran, N. (2019). Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99(3):032331.

- Schuld, M., Bocharov, A., Svore, K., and Wiebe, N. (2018). Circuit-centric quantum classifiers. *Physical Review A*, 98(3):032331.
- Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS 1994)*, pages 124–134. IEEE.
- Shor, P. W. (1997). Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509.
- Smith, S. L., Brock, A., Berrada, L., and De, S. (2023). Convnets match vision transformers at scale. *arXiv preprint arXiv:2310.16764*. arXiv preprint arXiv:2310.16764.